

SSK3313 Operating System

Quiz 5: Deadlock

Matric: 227154 Group: 3

Date: 17.12.2025

Name: Yue CHENGHAO

1. Consider the matrix tables below and answer the following questions:

	Claim matrix C			Allocation matrix A			Need matrix			Available vector V		
	R1	R2	R3	R1	R2	R3	R1	R2	R3	R1	R2	R3
A	4	7	3	3	4	2	1	3	1	1	0	1
B	1	2	2	1	2	0	0	0	2			
C	2	9	7	1	4	4	1	5	3			
D	0	1	2	0	1	1	0	0	1			

Using the deadlock avoidance strategy, lead the system into safe state by ensuring that the system of processes and resources is always in a safe state.

i. Compute the resources each process might still request and write your answer in the Need matrix above

Need = claim – allocation

ii. Show your steps by computing the available vector V after each process runs to completion.

Initial available vector: $v = [1, 0, 1]$

Step 1:

Process D can run since $\text{Need}(D) = [0, 0, 1] < V$,

After D: $v = [1, 0, 1] + [0, 1, 1] = [1, 1, 2]$

Step 2:

Process B can run since $\text{Need}(B) = [0, 0, 2] < V$,

After B: $V = [1, 1, 2] + [1, 2, 0] = [2, 3, 2]$

Step 3:

Process A can run since $\text{Need}(a) = [1, 3, 1] < V$,

AFTER a: $V = [2, 3, 2] + [3, 4, 2] = [5, 7, 4]$

Step 4:

Process C can run since $\text{Need}(C) = [1, 5, 3] < V$,

After C, $V = [5, 7, 4] + [1, 4, 4] = [6, 11, 8]$

iii. State the order of execution of the processes (i.e. which process should run 1st, 2nd, 3rd and last) that will ensure the system stays in a safe state.

D-B-A-C

[6 m]

2. Consider the matrix tables below and answer the following questions:

Request matrix Q					
	R1	R2	R3	R4	R5
P1	0	2	1	0	0
P2	0	0	1	2	2
P3	1	0	0	0	1
P4	2	0	1	0	1
P5	0	0	1	1	0

Allocation matrix A					
	R1	R2	R3	R4	R5
P1	1	0	3	0	1
P2	0	0	0	0	0
P3	2	0	1	1	1
P4	0	0	0	0	0
P5	1	0	0	0	0

Available vector V					
	R1	R2	R3	R4	R5
	0	0	1	1	2

Apply the steps of deadlock detection to the data above. Is a deadlock detected? If yes, state the deadlocked process(es).

Initial Work Vector (Available) = [0, 0, 1, 1, 2]

Finish Vector = {P1: F, P2: F, P3: F, P4: F, P5: F}

1. Check P5: Request [0, 0, 1, 1, 0] <= Work [0, 0, 1, 1, 2] . (True)
 - P5 runs and releases Allocation [1, 0, 0, 0, 0] .
 - New Work = [1, 0, 1, 1, 2]
 - Finish = {P5: T}
 -
2. Check P3: Request [1, 0, 0, 0, 1] <= Work [1, 0, 1, 1, 2] . (True)
 - P3 runs and releases Allocation [2, 0, 1, 1, 1] .
 - New Work = [3, 0, 2, 2, 3]
 - Finish = {P5: T, P3: T}
 -
3. Check P2: Request [0, 0, 1, 2, 2] <= Work [3, 0, 2, 2, 3] . (True)
 - P2 runs and releases Allocation [0, 0, 0, 0, 0] .
 - New Work = [3, 0, 2, 2, 3]
 - Finish = {P5: T, P3: T, P2: T}
 -
4. Check P4: Request [2, 0, 1, 0, 1] <= Work [3, 0, 2, 2, 3] . (True)
 - P4 runs and releases Allocation [0, 0, 0, 0, 0] .
 - New Work = [3, 0, 2, 2, 3]
 - Finish = {P5: T, P3: T, P2: T, P4: T}
 -
5. Check P1: Request [0, 2, 1, 0, 0] .
 - Requires 2 units of R2, but Available R2 is 0.
 - Condition [0, 2, 1, 0, 0] <= [3, 0, 2, 2, 3] is False.
 - P1 cannot execute.

Conclusion:

Is a deadlock detected? **Yes.**

State the deadlocked process(es): **P1**